



Domaine : *Sciences de l'Ingénieur*

Mention : *Informatique Réseaux et Télécommunication*

Spécialité : *Intelligence Artificielle et Big Data (IA/BD)*

DEVOIR FINAL: PIPELINE D'INGESTION
DE
DONNEES IOT EN TEMPS REEL
(SERVERLESS)

Rédigé par :

ADJASSEM Yao-Fiawomo Justin

Enseignant :

LOKOSSOU Hervé Mofiala

ANNEE ACADEMIQUE : 2025 - 2026

Lomé, Juin 2026

Table des matières

1. Questions theoriques	3
Question 1 : Infrastructure as Code et AWS CloudFormation	3
Question 2 : AWS Lambda et l'approche Serverless	3
Question 3 : CloudFront devant API Gateway pour l'IoT	3
Question 4 : S3 (Data Lake) + DynamoDB vs base relationnelle unique	4
Question 5 : Modele de responsabilite partagee pour S3	4
Question 6 : Static Website Hosting vs CloudFront + OAC.....	5
Question 7 : CloudWatch et la supervision serverless	5
Question 8 : Limites de Lambda pour les gros volumes	6
2. Demarche d'implementation	7
2.1 Structure du projet	7
2.2 Infrastructure (template.yaml).....	7
2.3 Fonction Lambda (main.py)	7
2.4 Script de test (test_client.py)	8
2.5 Documentation technique (index.html).....	8
3. Deploiement et validation	9
3.1 Deploiement CloudFormation (SAM)	9
3.2 Test d'ingestion — payload valide.....	9
3.3 Test d'ingestion — payload corrompu	10
3.4 Verification S3 (Data Lake)	11
3.5 Verification DynamoDB (Feature Store).....	13
3.6 Deploiement du site de documentation	14
3.7 Diagramme d'architecture (Mermaid)	15
3.8 Monitoring CloudWatch Logs	16

1. Questions theoriques

Question 1 : Infrastructure as Code et AWS CloudFormation

L'Infrastructure as Code (IaC) est une pratique qui consiste à définir et gérer l'ensemble de l'infrastructure informatique (serveurs, réseaux, bases de données, etc.) à travers des fichiers de configuration lisibles par l'homme, plutôt que par des manipulations manuelles via une interface graphique.

AWS CloudFormation est le service IaC natif d'Amazon Web Services. Il permet de décrire l'ensemble des ressources AWS dans un template au format YAML ou JSON. CloudFormation gère le cycle de vie complet de l'infrastructure : création, mise à jour et suppression des ressources de manière ordonnée, en respectant les dépendances entre elles. Il garantit l'idempotence (un même template produit toujours le même résultat), la reproductibilité (déploiement identique dans plusieurs régions ou comptes) et la traçabilité (chaque modification est versionnée). En cas d'erreur lors d'un déploiement, CloudFormation effectue automatiquement un rollback vers l'état précédent, assurant ainsi la cohérence de l'infrastructure.

Question 2 : AWS Lambda et l'approche Serverless

AWS Lambda est un service de calcul serverless qui exécute du code en réponse à des événements (requêtes HTTP, messages de file d'attente, modifications de base de données, etc.) sans qu'il soit nécessaire de provisionner ou gérer des serveurs.

Contrairement à une architecture classique basée sur Amazon EC2, où l'on doit dimensionner, configurer, patcher et surveiller des instances virtuelles en permanence, Lambda abstrait totalement la couche serveur. Les principales différences sont :

- Facturation à l'usage : on ne paie que le temps d'exécution réel (en millisecondes), tandis qu'une instance EC2 est facturée tant qu'elle est en fonctionnement, même au repos.
- Scalabilité automatique : Lambda s'adapte instantanément à la charge en créant autant d'instances parallèles que nécessaire, sans configuration d'Auto Scaling.
- Zéro administration : pas de système d'exploitation à gérer, pas de patches de sécurité à appliquer, pas de capacité à planifier.
- Modèle événementiel : Lambda est déclenché par des événements, ce qui s'intègre naturellement dans des architectures découplée et réactives.

Question 3 : CloudFront devant API Gateway pour l'IoT

Adosser une distribution Amazon CloudFront devant Amazon API Gateway pour la collecte de données IoT présente plusieurs avantages architecturaux majeurs :

- Réduction de la latence : CloudFront dispose d'un réseau mondial de plus de 400 points de présence (Edge Locations). Les capteurs IoT dispersés géographiquement se connectent au point de présence le plus proche, réduisant ainsi le temps de transit réseau.

- Protection DDoS : CloudFront intègre nativement AWS Shield Standard, offrant une protection contre les attaques par déni de service distribué, ce qui est critique pour des endpoints IoT exposés sur Internet.
- Terminaison TLS en périphérie : le chiffrement SSL/TLS est géré au niveau des Edge Locations, déchargeant API Gateway de cette tâche et améliorant les performances.
- Haute disponibilité : le réseau CloudFront assure une disponibilité supérieure en absorbant les pics de trafic avant qu'ils n'atteignent l'API Gateway.

Question 4 : S3 (Data Lake) + DynamoDB vs base relationnelle unique

Dans un écosystème Big Data, on privilégie une architecture à deux couches de stockage plutôt qu'une base relationnelle unique pour plusieurs raisons :

Amazon S3 comme Data Lake permet de stocker les données brutes dans leur format natif (JSON, CSV, Parquet, etc.) à un coût extrêmement faible (environ 0,023 \$/Go/mois). S3 offre une durabilité de 99,999999999 %, une capacité de stockage virtuellement illimitée et la compatibilité avec les outils analytiques (Athena, Spark, Redshift Spectrum) pour des requêtes ad hoc à posteriori.

Amazon DynamoDB comme Serving Layer (Feature Store) offre des temps de réponse inférieurs à 10 ms à n'importe quelle échelle, grâce à son modèle clé-valeur entièrement managé. Il est idéal pour servir des indicateurs agrégés en temps réel aux tableaux de bord ou aux applications analytiques.

Une base relationnelle unique (RDS, Aurora) ne pourrait pas absorber efficacement des flux massifs d'ingestion brute tout en servant des requêtes analytiques rapides. Elle imposerait un schéma rigide inadapté à la variété des données IoT, et son coût de stockage serait nettement supérieur à celui de S3 pour de grands volumes.

Question 5 : Modèle de responsabilité partagée pour S3

Le modèle de responsabilité partagée d'AWS définit une répartition claire des obligations de sécurité entre AWS et le client :

Responsabilité d'AWS (sécurité DU cloud) : AWS est responsable de l'infrastructure physique (centres de données, réseau, matériel), du stockage sous-jacent, de la disponibilité du service S3, de la durabilité des données (réplication automatique sur plusieurs zones de disponibilité) et du chiffrement côté serveur si activé.

Responsabilité du client (sécurité DANS le cloud) : le client est responsable de la configuration des accès à ses buckets (politiques de bucket, ACL, Block Public Access), de la gestion des identités et permissions (IAM), de l'activation et de la gestion du chiffrement (côté serveur SSE-S3/SSE-KMS ou côté client), de la classification de ses données et de la mise en place de la journalisation des accès (S3 Access Logs, CloudTrail).

En résumé, AWS garantit que le service S3 est fiable et sécurisé au niveau infrastructure, mais c'est au client de configurer correctement les permissions et le chiffrement pour protéger ses données.

Question 6 : Static Website Hosting vs CloudFront + OAC

Activer le Static Website Hosting public sur un bucket S3 est fortement déconseillé pour une documentation interne car cela impose de rendre le bucket publiquement accessible. Toute personne disposant de l'URL peut accéder aux fichiers, et il n'y a aucun mécanisme natif de contrôle d'accès granulaire, de journalisation avancée ou de protection contre les abus. De plus, le endpoint S3 Website ne supporte pas HTTPS nativement, exposant les données en transit à des interceptions.

L'utilisation de CloudFront avec un Origin Access Control (OAC) améliore considérablement la sécurité :

- Le bucket reste entièrement privé (Block Public Access active) : aucun accès direct n'est possible.
- Seule la distribution CloudFront, authentifiée par signature SigV4, peut lire les objets du bucket.
- Le trafic est chiffré de bout en bout via HTTPS (TLS).
- CloudFront offre des protections supplémentaires : AWS WAF (pare-feu applicatif), AWS Shield (anti-DDoS), géo-restriction et journalisation des accès.
- L'OAC remplace l'ancien mécanisme OAI (Origin Access Identity) avec une sécurité renforcée et une meilleure compatibilité avec les politiques de bucket modernes.

Question 7 : CloudWatch et la supervision serverless

Amazon CloudWatch est le service de supervision natif d'AWS. Pour les applications serverless basées sur Lambda, il joue un rôle central :

- CloudWatch Logs : chaque invocation Lambda génère automatiquement des logs dans un groupe de logs dédié (/aws/lambda/<nom-de-la-fonction>). On y retrouve les messages de debug (print/logging), les rapports d'exécution (durée, mémoire utilisée, etc.) et les traces d'erreurs complètes.
- CloudWatch Metrics : Lambda publie automatiquement des métriques (nombre d'invocations, durée, erreurs, throttles, invocations concurrentes) exploitables pour créer des tableaux de bord et des alarmes.
- CloudWatch Alarms : on peut configurer des alarmes sur les métriques (par exemple, alerte si le taux d'erreur dépasse 5 %) pour être notifié via SNS.

Lorsqu'une fonction Lambda lève une exception non gérée, le runtime Lambda capture automatiquement le stack trace Python complet et l'écrit dans CloudWatch Logs. L'invocation est marquée comme échouée, le compteur d'erreurs Lambda est incrémenté, et la réponse

HTTP retournée est un code 500 (Internal Server Error). Le stack trace permet d'identifier précisément la ligne de code et le type d'exception à l'origine de l'échec.

Question 8 : Limites de Lambda pour les gros volumes

Si un fichier de 50 Go est déposé d'un seul coup, l'architecture Lambda actuelle atteint ses limites pour plusieurs raisons :

- Timeout maximum : une fonction Lambda ne peut s'exécuter que pendant 15 minutes au maximum, ce qui est insuffisant pour traiter 50 Go de données.
- Mémoire limitée : Lambda est plafonnée à 10 Go de RAM, rendant impossible le chargement intégral d'un fichier de 50 Go en mémoire.
- Taille du payload : API Gateway limite les payloads à 10 Mo (REST API) ou 6 Mo (HTTP API), bien en dessous de 50 Go.
- Stockage éphémère : le répertoire /tmp de Lambda est limité à 10 Go.

Pour traiter ce volume, le service manage AWS le plus adapté est Amazon EMR (Elastic MapReduce). EMR permet d'exécuter des frameworks Big Data distribués comme Apache Spark, Hadoop ou Presto sur des clusters de machines scalables. Spark peut lire directement le fichier de 50 Go depuis S3, le partitionner automatiquement entre les nœuds du cluster et le traiter en parallèle en quelques minutes. Une alternative serait AWS Glue, le service ETL serverless d'AWS, qui repose également sur Spark mais sans gestion de cluster.

2. Démarche d'implémentation

2.1 Structure du projet

Le projet est organisé selon la structure suivante, déployée via AWS SAM (Serverless Application Model) :

```
EXAM_PROJECT/
  infrastructure/
    template.yaml          # Template SAM (toutes les ressources AWS)
  src/
    main.py                # Handler Lambda (ingestion IoT)
    requirements.txt        # Dependances Lambda (boto3)
  docs/
    index.html              # Site de documentation technique
    test_client.py          # Script de test client
    Makefile                # Automatisation build/deploy
    requirements.txt        # Dependances projet
    .gitignore
```

2.2 Infrastructure (template.yaml)

Le template SAM définit l'ensemble de l'infrastructure en une seule pile CloudFormation. Les ressources créées sont :

Ressource	Type AWS	Role
DataLakeBucket	S3::Bucket	Stockage brut (Data Lake)
FeatureStoreTable	DynamoDB::Table	Feature Store analytique
LambdaExecutionRole	IAM::Role	Permissions Lambda (S3 + DynamoDB + Logs)
IngestionFunction	Serverless::Function	Lambda Python 3.11
HttpApi	ApiGatewayV2::Api	Point d'entree HTTP
IngestionCloudFront	CloudFront::Distribution	CDN devant l'API
IngestionCachePolicy	CloudFront::CachePolicy	Cache desactive (TTL=0)
DocBucket	S3::Bucket	Bucket prive documentation
DocOAC	CloudFront::OriginAccessControl	Controle d'accès OAC SigV4
DocCloudFront	CloudFront::Distribution	CDN securise pour la doc

2.3 Fonction Lambda (main.py)

La fonction Lambda réalise les opérations suivantes à chaque requête POST :

1. Parse le corps JSON de la requête (sensor_id + readings).
2. Génère un identifiant unique (UUID v4) et un horodatage UTC.
3. Sauvegarde le payload brut dans S3 avec partitionnement temporel : raw-zone/year=YYYY/month=MM/{sensor_id}_{uuid}.json.

4. Calcule la température moyenne et le nombre de mesures en statut ERROR.
5. Enregistre un rapport condense dans DynamoDB (request_id, timestamp, sensor_id, s3_path, avg_temperature, error_count, total_readings).
6. Retourne une réponse HTTP 201 avec le résumé de l'ingestion.

2.4 Script de test (test_client.py)

Le script test_client.py prend en argument l'URL CloudFront d'ingestion et exécute deux tests :

- Test 1 (payload valide) : envoie 5 mesures structurées avec des températures et statuts (dont 2 ERROR). Attend une réponse HTTP 201.
- Test 2 (payload corrompu) : envoie un reading sans clé température pour provoquer une KeyError dans la Lambda. Attend une réponse HTTP 500.

2.5 Documentation technique (index.html)

Une page HTML responsive a été créée dans docs/index.html. Elle contient la description complète de l'architecture, les services utilisés, le schéma DynamoDB, le partitionnement S3, et la sécurisation via OAC. Elle est uploadée dans le bucket S3 de documentation et accessible uniquement via CloudFront.

Un diagramme d'architecture interactif a été intégré directement dans la page HTML grâce à la bibliothèque Mermaid.js. Ce diagramme illustre le flux complet des données : depuis les capteurs IoT, en passant par CloudFront, API Gateway et Lambda, jusqu'au stockage dans S3 et DynamoDB. Il représente également le flux secondaire de la documentation (Utilisateur -> CloudFront OAC -> S3 privé) ainsi que le monitoring via CloudWatch Logs.

3. Deploiement et validation

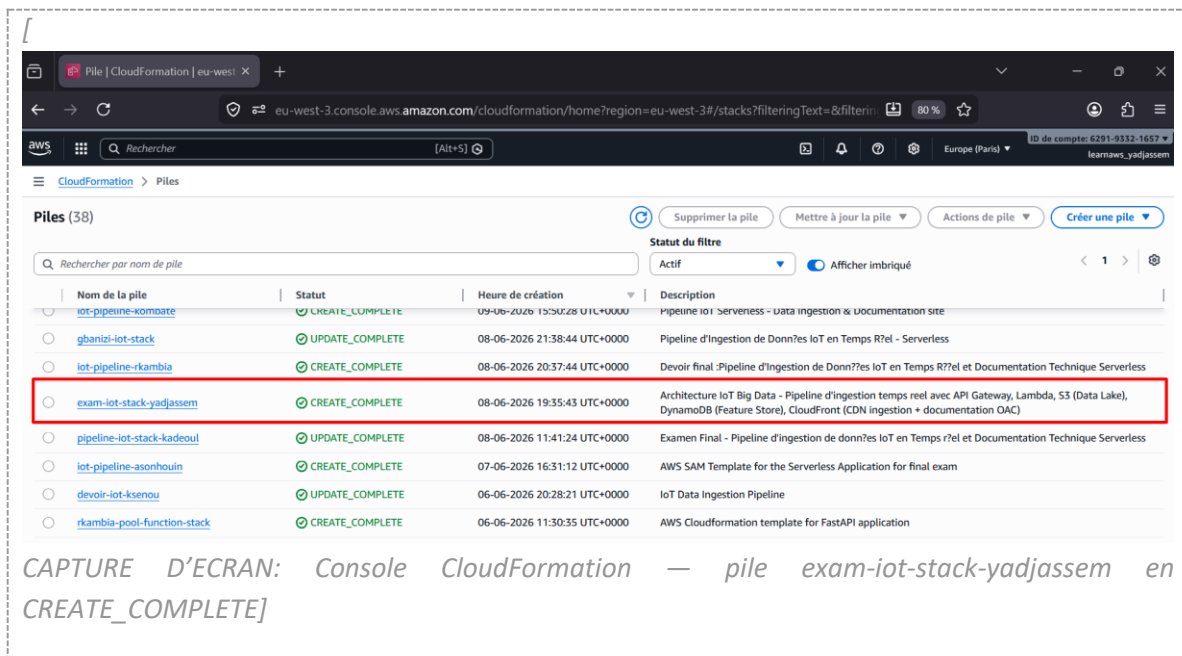
3.1 Deploiement CloudFormation (SAM)

La pile a ete deployee via la commande 'make deploy' qui execute sam build (avec Docker pour Python 3.11) puis sam deploy. Le statut final est CREATE_COMPLETE.

Commandes executees :

```
# Build avec conteneur Docker (Python 3.11)
sam build --use-container --region eu-west-3 --template-file
infrastructure/template.yaml
```

```
# Deploiement de la stack
sam deploy --resolve-s3 --region eu-west-3 \
--stack-name exam-iot-stack-yadjassem \
--capabilities CAPABILITY_NAMED_IAM
```

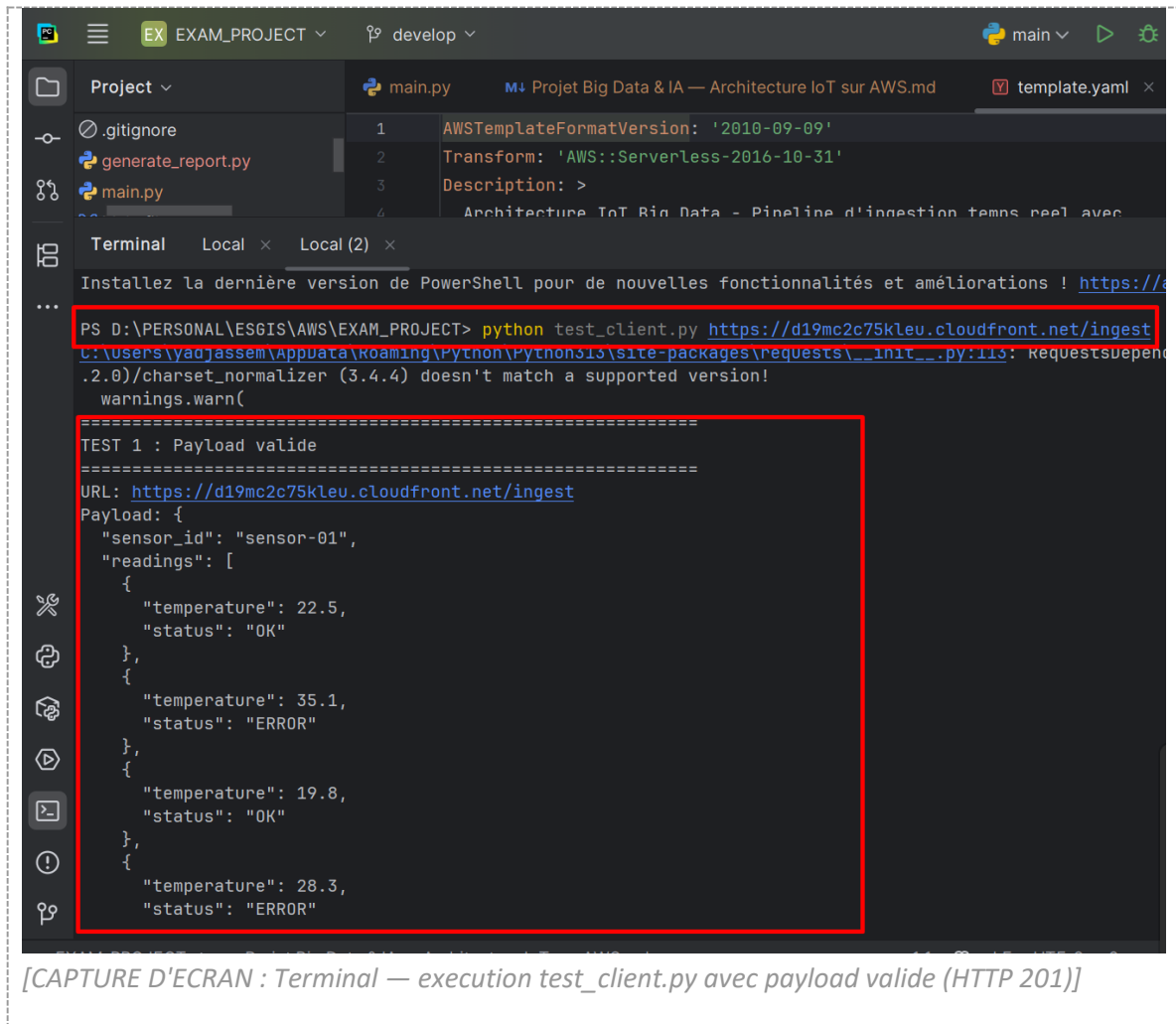


3.2 Test d'ingestion — payload valide

Commande executee :

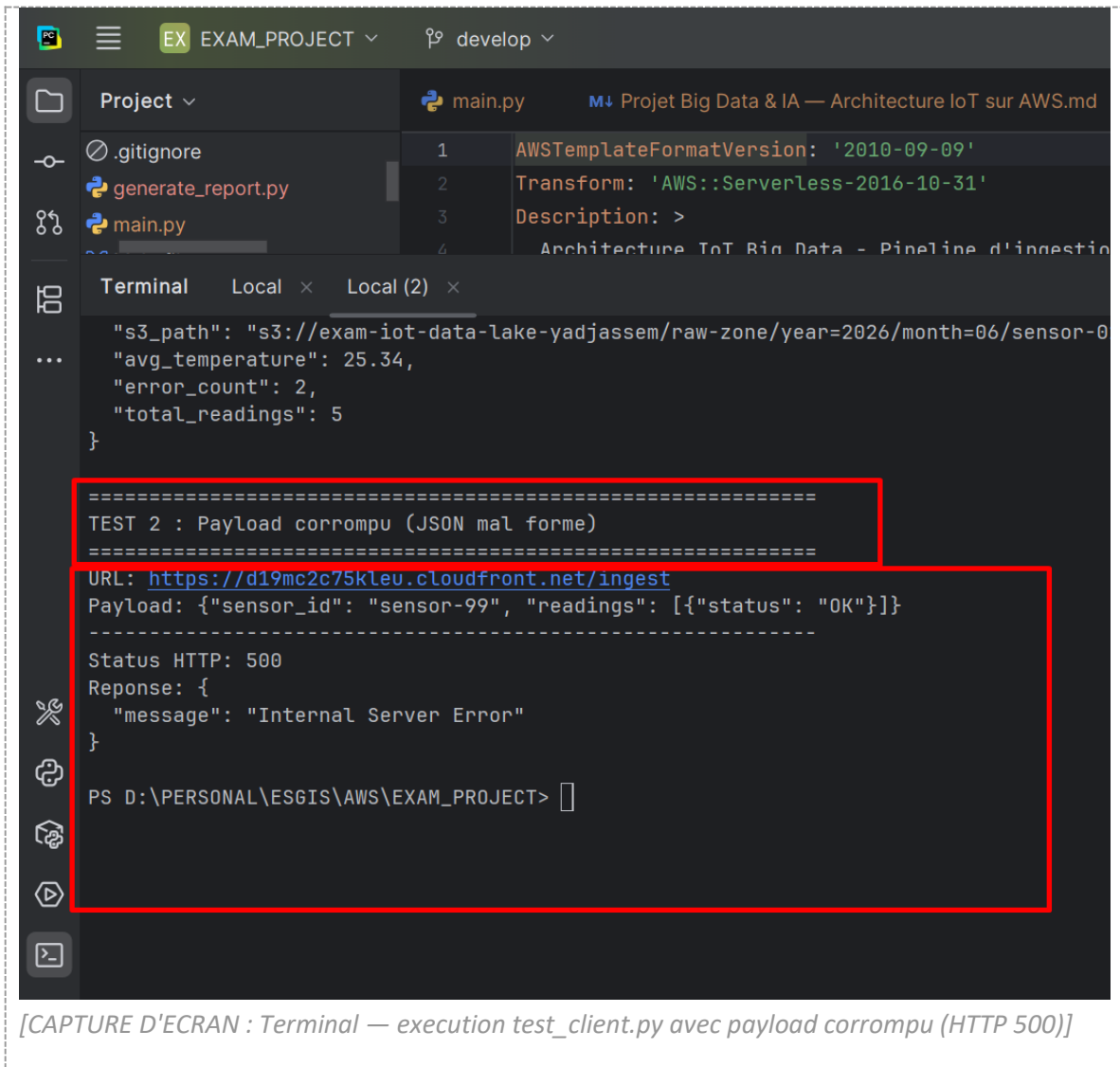
```
python test_client.py https://d19mc2c75k1eu.cloudfront.net/ingest
```

Exécution du script test_client.py avec un payload contenant 5 mesures structurées (sensor_id, temperature, status dont 2 en ERROR). La réponse HTTP 201 confirme le bon fonctionnement du pipeline complet.



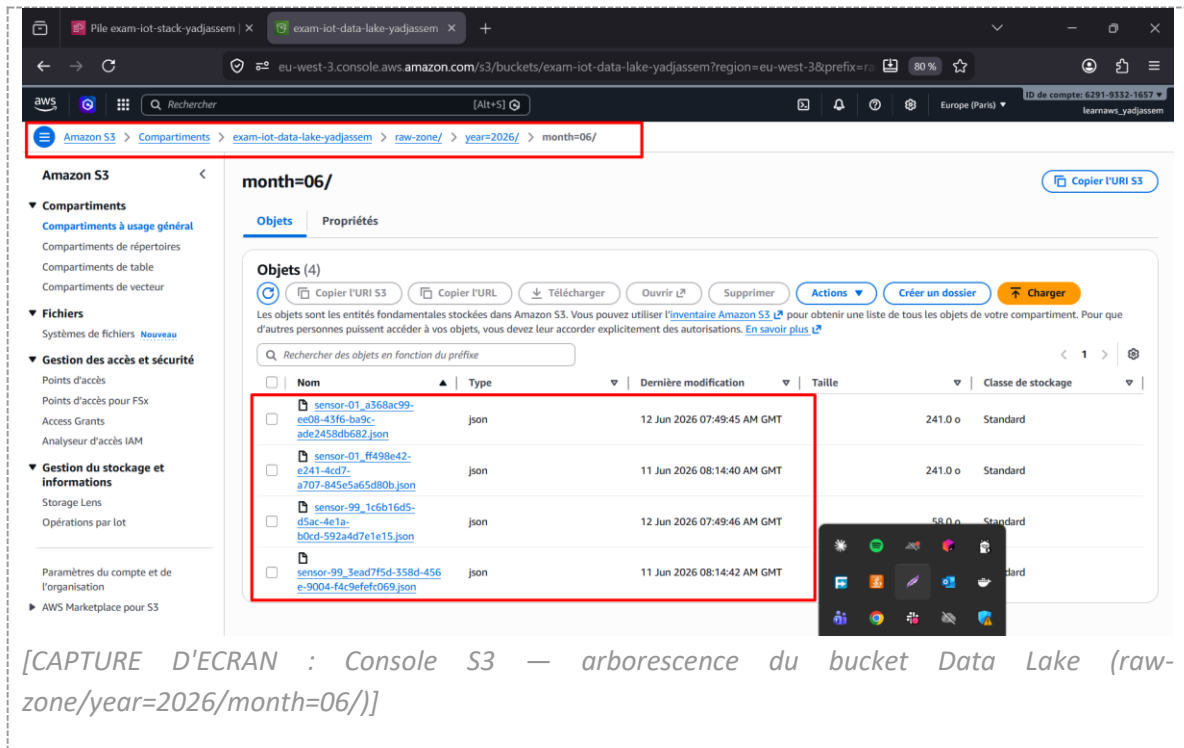
3.3 Test d'ingestion — payload corrompu

Envoi d'un payload sans cle temperature pour provoquer une erreur. La Lambda retourne HTTP 500 (Internal Server Error).



3.4 Verification S3 (Data Lake)

Verification dans la console S3 de la presence du fichier JSON dans le bucket exam-iot-data-lake-yadjassem avec l'arborescence partitionnee raw-zone/year=2026/month=06/.

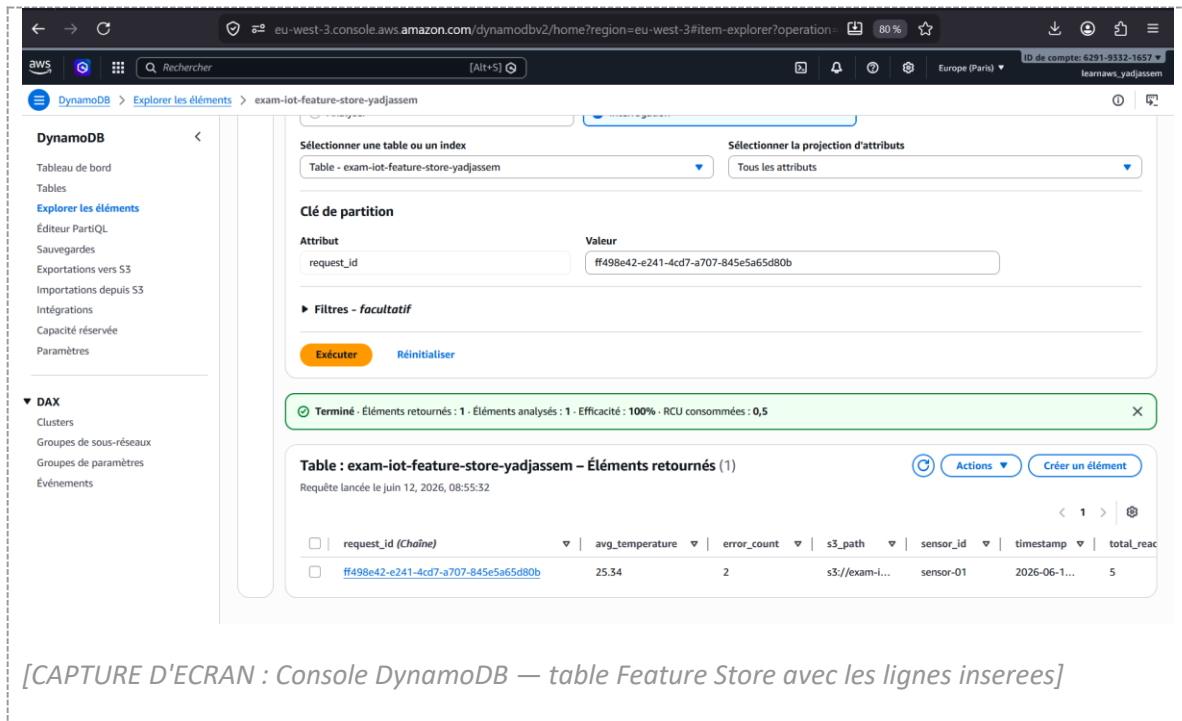


```
sensor-01_a368ac99-ee08-43f6-ba9c-ade2458db682.json X
C: > Users > yadjassem > Downloads > sensor-01_a368ac99-ee08-43f6-ba9c-ade2458db682.json > abc sensor_id
1 {
2   "sensor_id": "sensor-01",
3   "readings": [
4     {"temperature": 22.5, "status": "OK"},
5     {"temperature": 35.1, "status": "ERROR"},
6     {"temperature": 19.8, "status": "OK"},
7     {"temperature": 28.3, "status": "ERROR"},
8     {"temperature": 21.0, "status": "OK"}
9   ]
10 }
```

[CAPTURE D'ECRAN : Console S3 — contenu du fichier JSON brut]

3.5 Verification DynamoDB (Feature Store)

Verification dans la console DynamoDB de l'insertion des metriques agregees dans la table exam-iot-feature-store-yadjassem.

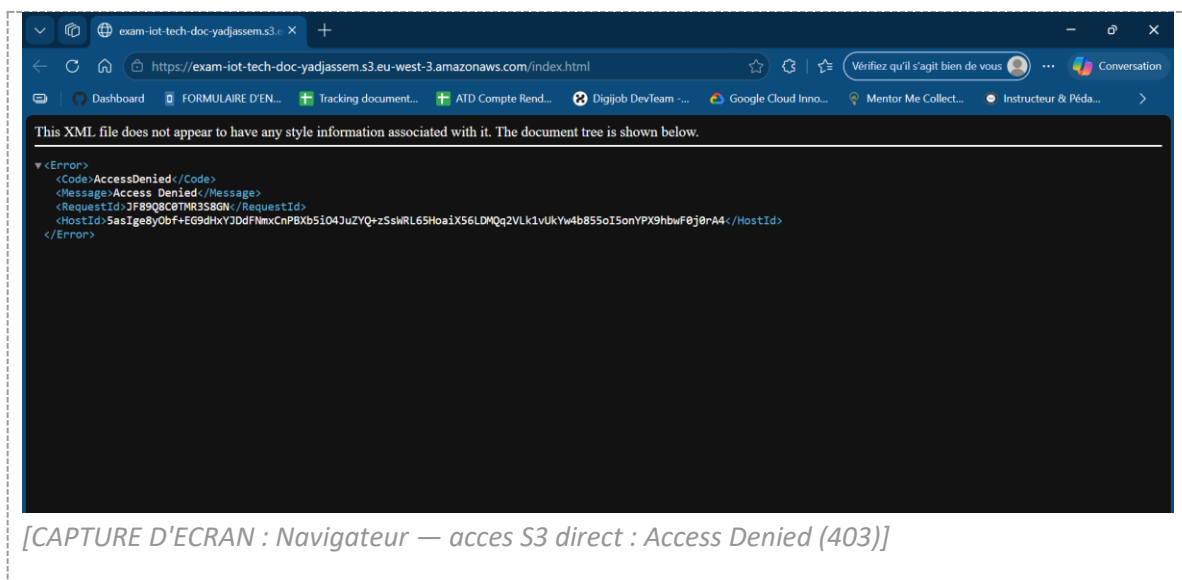


3.6 Deploiement du site de documentation

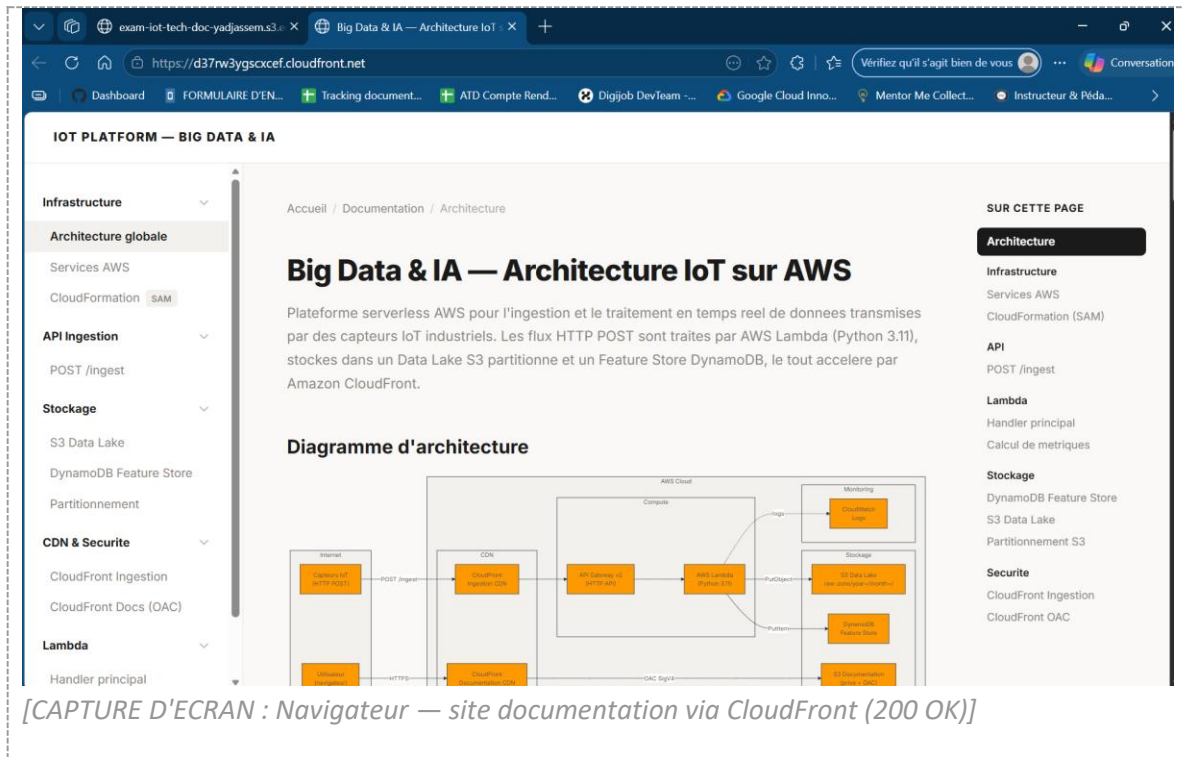
Le fichier index.html a été uploadé via la commande CLI :

```
aws s3 cp docs/index.html s3://exam-iot-tech-doc-yadjassem/index.html --region eu-west-3
```

Test d'accès direct au bucket S3 : Access Denied (403), confirmant que le bucket est bien privé.



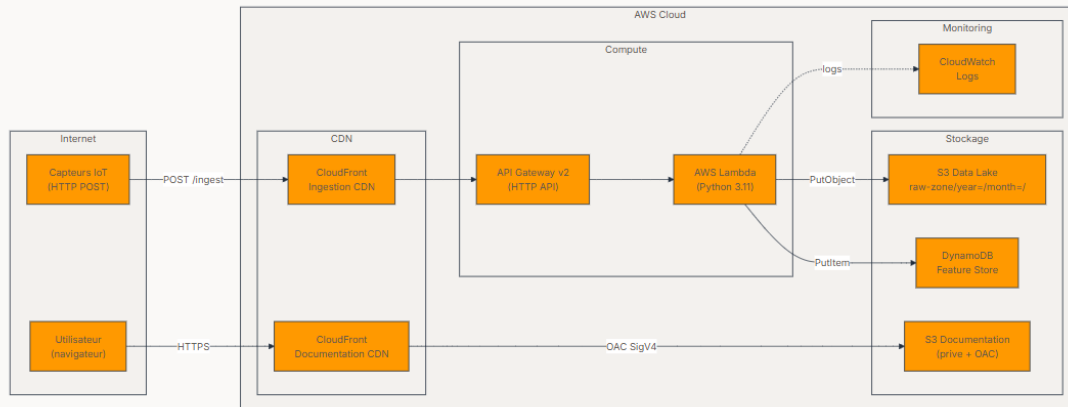
Test d'accès via CloudFront : la page s'affiche correctement, confirmant le bon fonctionnement de l'OAC.



3.7 Diagramme d'architecture (Mermaid)

Le diagramme d'architecture ci-dessous est généré dynamiquement dans la page de documentation technique via Mermaid.js. Il représente l'ensemble des flux de données et des services AWS impliqués dans l'architecture.

Diagramme d'architecture

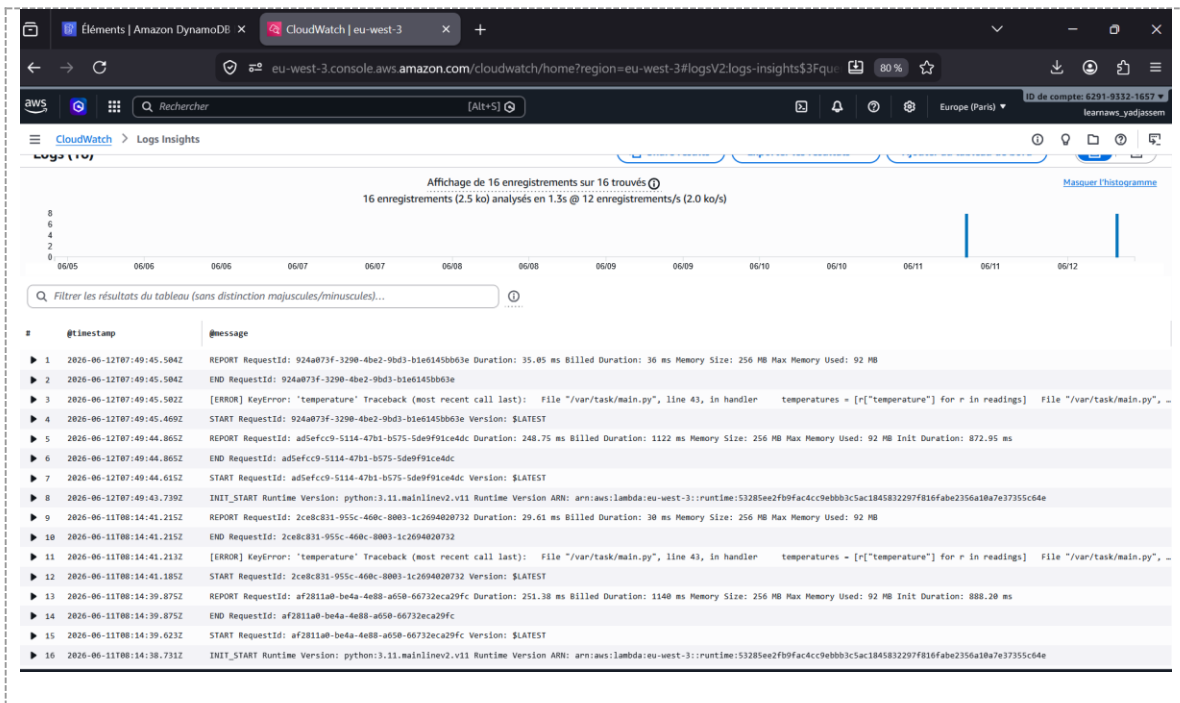


[CAPTURE D'ECRAN : Page documentation — diagramme d'architecture Mermaid (flux IoT + documentation)]

3.8 Monitoring CloudWatch Logs

Le groupe de logs /aws/lambda/exam-iot-ingestion-yadjassem contient les traces d'exécution de la fonction Lambda.

Execution reussie :



[CAPTURE D'ECRAN : CloudWatch Logs — trace d'une execution reussie (START, END, REPORT)]